

An Introduction to **Great Tables**

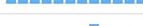


Display Tables... What Do We Want?

shape: (15, 5)

product	revenue_dollars	revenue_pct	margin_dollars	margin_pct
str	f64	f64	f64	f64
"Grinder"	904.5	0.03	567.96	0.04
"Moka pot"	2045.25	0.07	181.08	0.01
"Cold brew"	288.75	0.01	241.77	0.02
"Filter"	404.25	0.01	70.01	0.02
"Drip machine"	2632.0	0.1	1374.45	0.09
...	...	...	...	...
"Dripper"	575.75	0.02	139.02	0.01
"Scale"	3801.0	0.13	2910.29	0.19
"Kettle"	756.25	0.02	617.52	0.04
"Espresso Machi..."	8406.0	0.28	3636.44	0.24
"Total"	30284.25	1.0	14932.16	1.0

Less of This

Sales of Coffee Equipment						
icon	product	Revenue		Profit		monthly_sales
		Earnings	Fraction	Earnings	Fraction	
	Grinder	\$904K	3%	\$568K	4%	
	Moka pot	\$2.05M	7%	\$181K	1%	
	Cold brew	\$289K	1%	\$242K	2%	
	Filter	\$404K	1%	\$70.0K	0%	
	Drip machine	\$2.63M	9%	\$1.37M	9%	
	AeroPress	\$2.60M	9%	\$1.29M	9%	
	Pour over	\$846K	3%	\$365K	2%	
	French press	\$1.11M	4%	\$748K	5%	
	Cezve	\$2.51M	9%	\$1.97M	13%	
	Chemex	\$3.14M	11%	\$818K	6%	
	Scale	\$3.80M	13%	\$2.91M	20%	
	Kettle	\$756K	3%	\$618K	4%	
	Espresso Machine	\$8.41M	29%	\$3.64M	25%	
Total		\$29.4M	102%	\$14.8M	100%	

More of This

We Want the Tables to Look Good and be Reproducible.

History does not bode well for the Hoosiers

Only one future tournament team made fewer 3PTs through their first six games than Indiana in 2024.

TEAM	Shooting				SEED	ROUND	YEAR
	3FG	3FG%	PER GAME				
WF Wake Forest	17-61	27.87%	2.83		4	R64	2009
Indiana	19-79	24.05%	3.17	???	???	2024	
North Carolina	20-60	33.33%	3.33	6	R32	2014	
CSU Coppin St.	21-60	35.00%	3.50	16	R68	2008	
Vermont	22-89	24.72%	3.67	16	R64	2010	
New Mexico St.	22-72	30.56%	3.67	13	R64	2014	

Viz. + Analysis by @andreweatherman

2023 Median CO2 Intensity (gCO2eq/kWh) and Power Consumption Breakdown (%)														
Zone	CO2 Intensity	Hydro	Nuclear	Wind	Solar	Geothermal	Biomass	Gas	Coal	Oil	Unknown	Hydro Discharge	Battery Discharge	
Sweden	22	43.5%	29.2%	23.4%	0.9%	0%	0.2%	0.2%	0%	4.4%	0%	0%	0%	
Iceland	28	72.4%	0%	0%	0%	27.6%	0%	0%	0%	0%	0%	0%	0%	
Quebec	30	91.5%	0.9%	4.7%	0%	0%	2.5%	0.4%	0%	0%	0%	0%	0%	
France	44	10.5%	54.1%	10.4%	4.8%	0%	1.4%	6.6%	0.5%	0.3%	0.1%	1.1%	0%	
Tasmania	47	86.3%	0%	19.2%	5.8%	0%	0%	0.6%	8.3%	0%	0%	0%	0%	
Ontario	70	26%	51.5%	8.1%	0.5%	0%	0.2%	13.3%	0%	0%	0%	0%	0%	
Finland	79	24.6%	40.9%	18.6%	0.8%	0%	6.6%	2.2%	5.2%	0%	1.1%	0%	0%	
New Zealand	89	62.8%	0%	7.4%	0%	18.2%	0%	6.1%	3.6%	0%	2%	0%	0%	
West Denmark	123	18.2%	4.1%	45.7%	8.1%	0%	7.5%	5.9%	8.2%	0.4%	0.6%	0.3%	0%	
Belgium	125	1.4%	52.1%	19.7%	9.9%	0%	3.3%	18.7%	1.5%	0.1%	2.1%	1.4%	0%	
East Denmark	140	13.1%	10.5%	40.7%	6.6%	0%	14.3%	4.2%	7.2%	1.2%	2.2%	0.1%	0%	
South Australia	145	1%	0%	45%	24.5%	0%	0%	21.8%	7%	0.1%	0%	0%	0.5%	
Spain	146	10%	22.7%	23.7%	15%	0%	2.1%	21.8%	1.6%	0.2%	0.3%	2.5%	0%	
Great Britain	200	4.6%	18.1%	29.2%	6%	0%	5.1%	33.2%	1.2%	0%	1%	0.6%	0%	
California	261	12.4%	10%	9.6%	17.1%	3.1%	1.7%	42%	1.2%	0%	0.9%	0%	1.9%	
Netherlands	261	3%	4.9%	31.5%	16.8%	0%	5%	27.7%	9%	0.8%	1.1%	0.2%	0%	
New York ISO	276	22.6%	22.7%	3.9%	0.1%	0%	0.1%	48.3%	0.6%	0%	1.8%	0%	0%	
Italy (North)	305	24.5%	11.9%	2.9%	6.9%	0.3%	2.2%	17.1%	2.6%	0.2%	8.7%	2.8%	0%	
Germany	395	5.9%	4.3%	23.7%	11.8%	0%	9.6%	11.3%	25.3%	0.5%	0.7%	2.3%	0%	
Ireland	397	2.6%	1.1%	36%	0.4%	0%	2.5%	46.2%	9.5%	1.4%	0.1%	0%	0%	
Texas	392	0.1%	9.1%	25.2%	7.2%	0%	0%	44.4%	13.8%	0%	0.3%	0%	0%	
Alberta	440	2.9%	0%	11.4%	2.7%	0%	2.6%	67.5%	8.1%	0%	4.6%	0%	0%	
Western Australia	450	0%	0%	15.5%	19.1%	0%	0.4%	35.1%	29.7%	0%	0%	0%	0.1%	
Victoria	511	6.3%	0%	20.6%	12.7%	0%	0%	14%	59.8%	0%	0%	0%	0.2%	
India (North)	547	21.3%	2.2%	1.5%	7.7%	0%	0%	1.9%	64.3%	0%	1.2%	0%	0%	
New South Wales	604	4.9%	0%	0%	19.6%	0%	0.1%	2.2%	64%	0%	0%	0%	0.1%	
Queensland	681	2.1%	0%	4%	19.8%	0%	0.2%	6.7%	67.1%	0%	0%	0%	0.1%	
South Africa	703	0.9%	4.2%	5.7%	3.2%	0%	0%	0%	80.9%	2.6%	0.1%	2.3%	0%	
Poland	763	2.6%	1.4%	15.2%	8.5%	0%	1.8%	8.5%	59.2%	1.5%	0.2%	1.1%	0%	

Source: (E.ON Climate & Renewables) Germany: <https://www.eonclimate.com/en>; Netherlands: <https://www.eonclimate.com/en>; France: <https://www.eonclimate.com/en>; Spain: <https://www.eonclimate.com/en>; Italy: <https://www.eonclimate.com/en>; Germany: <https://www.eonclimate.com/en>; California: <https://www.eonclimate.com/en>; Texas: <https://www.eonclimate.com/en>; Alberta: <https://www.eonclimate.com/en>; Western Australia: <https://www.eonclimate.com/en>; Victoria: <https://www.eonclimate.com/en>; India: <https://www.eonclimate.com/en>; New South Wales: <https://www.eonclimate.com/en>; Queensland: <https://www.eonclimate.com/en>; South Africa: <https://www.eonclimate.com/en>; Poland: <https://www.eonclimate.com/en>.
Some emissions factors are based on IPCC 2014 defaults, while some are based on more accurate regional factors. All zones are publicly available on the Carbon Intensity and Emissions Factors tool via Google data link.

Player Report Card: Kyle Filipowski

Position groups are determined by Barttorvik. Data through Jan 16, 2024
Percentile relative to bigs across...



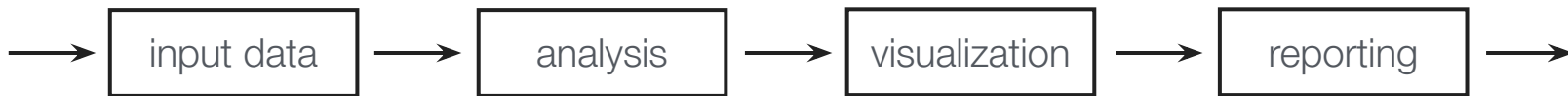
STAT	VALUE	NATIONAL	HIGH-MAJORS	CONF. RANK
Box (Per-Game)				
Minutes	29.8	92%	86%	28
Points	18.2	98%	97%	5
Off. Rebounds	2.3	62%	51%	9
Def. Rebounds	6.5	94%	91%	2
Turnovers	1.7	30%	28%	54
Shooting				
eFG%	56.7%	51%	35%	22
3FG%	41.2%	84%	84%	21
FT%	68.4%	66%	65%	56
3FGA/FGA ¹	24.4	87%	89%	68
Advanced				
Off. Rating ²	119.9	83%	66%	17
Def. Rating	88.9	93%	89%	2
Usage	28.8	94%	97%	5
PORPAG ³	5.1	99%	98%	2

¹ 3FG attempts per 100 FGA

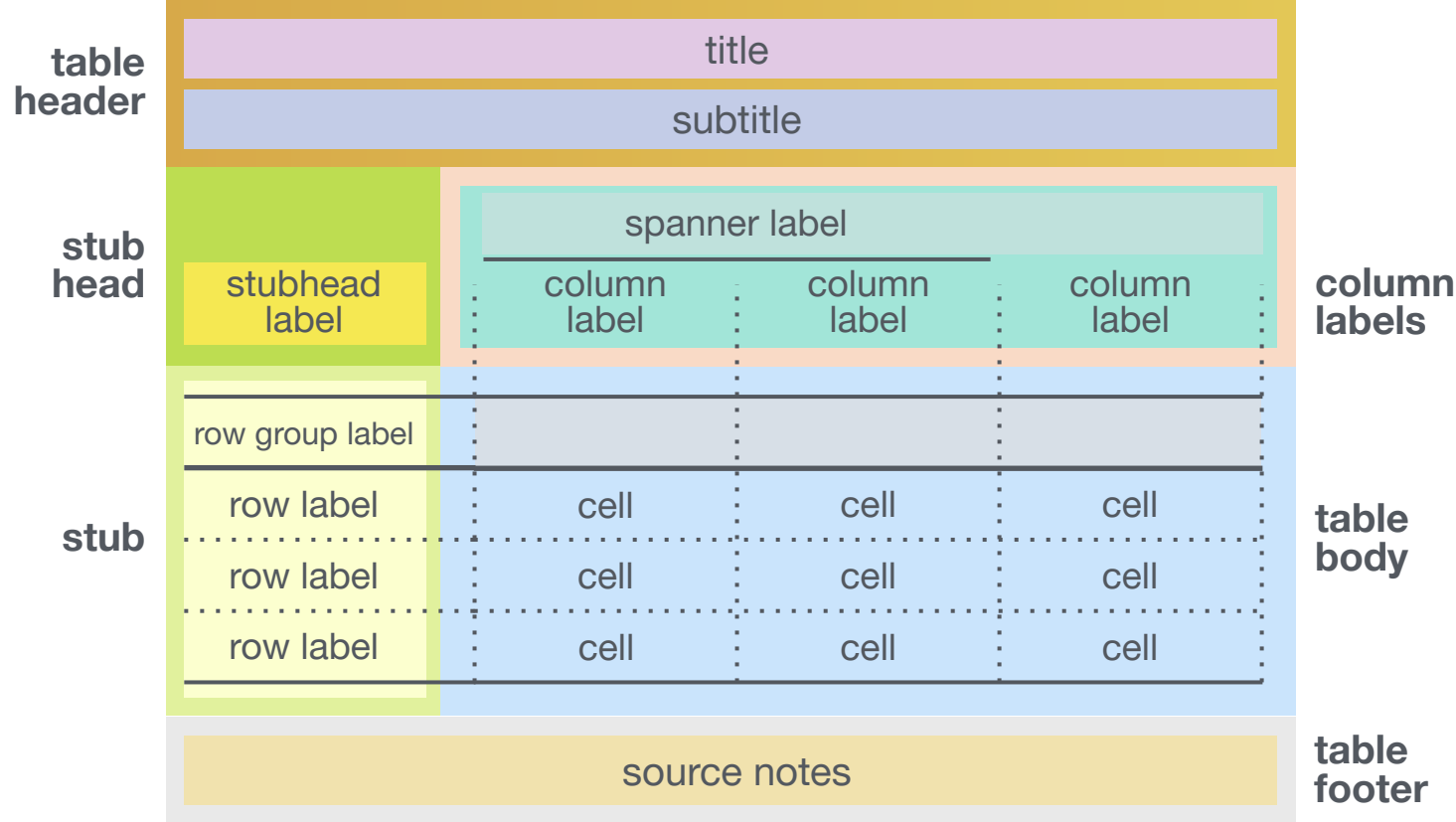
² Offensive and defensive ratings represent points scored/allowed per 100 possessions

³ PORPAG represents points above replacement player at that usage

We benefit from a reproducible workflow.



The Table Display Framework



Great Tables



The **Great Tables** package is focused purely on the display of tables.

It's not the only approach in Python, but it is:

- comprehensive
- actively-developed
- attentive to all table-related problems

Let's now go through some of the basics on using the package to make tables.

Structure Basics

Structure: GT () — How to Begin with the GT API

We first need to introduce our data to **Great Tables**.

This could all start with either a Pandas or a Polars DataFrame.

Code

```
GT(<data>)
```

Structure: GT() — How to Begin with the GT API

Let's use an example dataset, **exibble**, to make a **Great Tables** table:

Code

```
from great_tables import GT, exibble  
  
GT(exibble)
```

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b

Structure: GT() — Making a Stub with Row Labels

See this column called **row**? It contains row labels and we could structure this table with a stub (holds row labels).

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b

Structure: GT() — Making a Stub with Row Labels

GT() has the **rowname_col=** arg. Supply the column name there and it moves into the stub.

Code

```
from great_tables import GT, exhibble

GT(exibble, rowname_col="row")
```

	num	char	fctr	date	time	datetime	currency	group
row_1	1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	grp_a
row_2	2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	grp_a
row_3	3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	grp_a
row_4	4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	grp_a
row_5	5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	grp_b
row_6	NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	grp_b
row_7	7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	grp_b
row_8	8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	grp_b

Structure: GT() — Making Row Groups

See this column called **group**? It contains categories for grouping rows, so and we could add row groups to the table.

	num	char	fctr	date	time	datetime	currency	group
row_1	1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	grp_a
row_2	2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	grp_a
row_3	3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	grp_a
row_4	4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	grp_a
row_5	5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	grp_b
row_6	NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	grp_b
row_7	7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	grp_b
row_8	8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	grp_b

Structure: GT() — Making Row Groups

Use the **groupname_col=** arg to define row groups with a column.

Code

```
GT(  
  exhibble,  
  rowname_col="row",  
  groupname_col="group"  
)
```

	num	char	fctr	date	time	datetime	currency
grp_a							
row_1	1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950
row_2	2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950
row_3	3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390
row_4	4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000
grp_b							
row_5	5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810
row_6		NA fig	six	2015-06-15	NA	2018-06-06 16:11	13.255
row_7	7.770e+05	grapefruit	seven		NA 19:10	2018-07-07 05:22	NA
row_8	8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440

Structure: Comparing a Basic and a More Structured Table

Basic GT Table

num	char	fctr	date	time	row	group
1.111e-01	apricot	one	2015-01-15	13:35	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	row_5	grp_b
	NA fig	six	2015-06-15	NA	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	row_8	grp_b

Table w/ Stub & Row Groups

	num	char	fctr	date
grp_a				
row_1	1.111e-01	apricot	one	2015-01-15
row_2	2.222e+00	banana	two	2015-02-15
row_3	3.333e+01	coconut	three	2015-03-15
row_4	4.444e+02	durian	four	2015-04-15
grp_b				
row_5	5.550e+03	NA	five	2015-05-15
row_6		NA fig	six	2015-06-15
row_7	7.770e+05	grapefruit	seven	NA
row_8	8.880e+06	honeydew	eight	2015-08-15

Structure: `tab_header()` — Adding a Title to Your Table

Adding a title to the GT table in a header component can be good for presentation. We do that with **`tab_header()`**.

Code

```
GT(exibble) \
  .tab_header(title="Table Title")
```



Table Title

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b

Structure: `tab_header()` — You Can Also Add a Subtitle

Having a subtitle is also an option here.

Code

```
GT(exibble) \
  .tab_header(
    title="Table Title",
    subtitle="The Subtitle"
  )
```

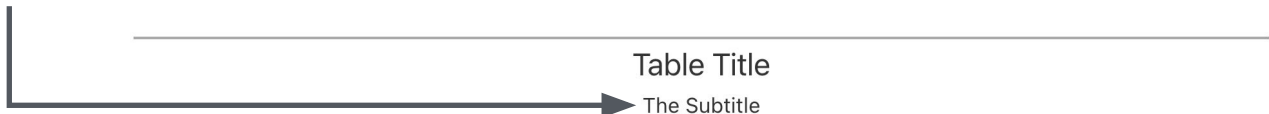


Table Title								
The Subtitle								
num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b

Structure: `tab_source_note()` — Adding Notes to Your Table

Adding notes to the GT table in a footer component can help explain the table. We do that with **tab_source_note()**.

Code

```
GT(exibble) \
.tab source note(source note="Note")
```

[illegible]

Structure: `cols_width()` — Adjusting the Widths of Columns

We can manually specify the widths of columns by using `cols_width()`.

Code

```
GT(exibble[["num", "char", "date"]]) \
  .cols_width(
    cases={
      "num": "150px",
      "char": "100px",
      "date": "200px"
    }
  )
```

num	char	date
1.111e-01	apricot	2015-01-15
2.222e+00	banana	2015-02-15
3.333e+01	coconut	2015-03-15
4.444e+02	durian	2015-04-15
5.550e+03	NA	2015-05-15
NA	fig	2015-06-15
7.770e+05	grapefruit	NA
8.880e+06	honeydew	2015-08-15

num	char	date
1.111e-01	apricot	2015-01-15
2.222e+00	banana	2015-02-15
3.333e+01	coconut	2015-03-15
4.444e+02	durian	2015-04-15
5.550e+03	NA	2015-05-15
NA	fig	2015-06-15
7.770e+05	grapefruit	NA
8.880e+06	honeydew	2015-08-15

Structure: `cols_width()` — Adjusting the Widths of Columns

Widths are determined by
content size.

num	char	date
1.111e-01	apricot	2015-01-15
2.222e+00	banana	2015-02-15
3.333e+01	coconut	2015-03-15
4.444e+02	durian	2015-04-15
5.550e+03	NA	2015-05-15
NA	fig	2015-06-15
7.770e+05	grapefruit	NA
8.880e+06	honeydew	2015-08-15

Default Widths



num	char	date
1.111e-01	apricot	2015-01-15
2.222e+00	banana	2015-02-15
3.333e+01	coconut	2015-03-15
4.444e+02	durian	2015-04-15
5.550e+03	NA	2015-05-15
NA	fig	2015-06-15
7.770e+05	grapefruit	NA
8.880e+06	honeydew	2015-08-15

Customized Widths

Structure: `tab_spanner()` — Spanners Above Column Labels

Another way to add structure is to add spanner labels above sets of column labels. We do this with **`tab_spanner()`**.

Code

```
GT(exibble) \
  .tab_spanner(
    columns=["date", "time", "datetime"],
    label="A Spanner"
  )
```



			A Spanner						
num	char	fctr	date	time	datetime	currency	row	group	
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a	
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a	
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a	
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a	

Structure: `cols_label()` — Making Column Labels Much Nicer

The column labels are derived from the column names. We usually want to make them more presentable and it's done with **`cols_label()`**.

Code

```
GT(exibble) \
  .cols_label(
    num="Numbers",
    char="Fruits"
  )
```



Numbers	Fruits	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.555e+03	kiwi	five	2015-05-15	17:55	2018-05-05 04:00	1005.010	row_5	grp_a

Formatting Basics

Format: the family of `fmt_*( )` Methods

There is a huge number of formatting methods!

They all begin with **fmt_** and they format values in the table body.

They typically operate on whole columns of data but you can subset the columns' rows.

Here's a sampling of what is available:

`fmt_number()`

`fmt_integer()`

`fmt_scientific()`

`fmt_engineering()`

`fmt_percent()`

`fmt_currency()`

`fmt_date()`

`fmt_time()`

`fmt_datetime()`

`fmt_markdown()`

`fmt_image()`

`fmt()`

|
+ many more.

Format: `fmt_currency()` — Formatting Monetary Values

Let's again use the example dataset, **exibble**, and see what we're starting with.

Code

```
from great_tables import GT, exibble

GT(exibble)
```

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b

Format: `fmt_currency()` — Formatting Monetary Values

There's a column called `currency` here. Let's format that with `fmt_currency()`

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b

Format: `fmt_currency()` — Formatting Monetary Values

We've got to specify the columns here when using any formatter. In this case it is the one column called **currency**.

Code

```
GT(exibble) \
  .fmt_currency(columns="currency")
```

num	char	fctr	date	time	datetime	currency	row	group	
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	\$49.95	row_1	grp_a	
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	\$17.95	row_2	grp_a	
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	\$1.39	row_3	grp_a	
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	\$65,100.00	row_4	grp_a	
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	\$1,325.81	row_5	grp_b	
	NA fig	six	2015-06-15	NA	2018-06-06 16:11	\$13.26	row_6	grp_b	
7.770e+05	grapefruit	seven		NA 19:10	2018-07-07 05:22	NA	row_7	grp_b	
8.880e+06	honeydew	eight	2015-08-15	20:20		NA	\$0.44	row_8	grp_b

Format: `fmt_currency()` — Formatting Monetary Values

Before Formatting

currency
49.950
17.950
1.390
65100.000
1325.810
13.255
NA
0.440

After Formatting

currency
\$49.95
\$17.95
\$1.39
\$65,100.00
\$1,325.81
\$13.26
NA
\$0.44

What changes can you see in the *before* and *after* of this formatting?

Format: `fmt_percent()` — Formatting as Percentages

There's not really a great column to demonstrate percentage formatting in **exibble**.
Let's make up a new table with five rows:

row	value
1	0.0200
2	0.4345
3	0.0520
4	0.7530
5	1.0234

Format: `fmt_percent()` — Formatting as Percentages

We will format that called **value** column with `fmt_percent()`. Let's use the defaults.

Code

```
GT(<data>) \
  .fmt_percent(columns="value")
```

row	value
1	2.00%
2	43.45%
3	5.20%
4	75.30%
5	102.34%

Format: `fmt_percent()` — Formatting as Percentages

Before Formatting

row	value
1	0.0200
2	0.4345
3	0.0520
4	0.7530
5	1.0234

After Formatting

row	value
1	2.00%
2	43.45%
3	5.20%
4	75.30%
5	102.34%

What changes can you see in the *before* and *after* of this formatting?

Styling basics

Style: `tab_style()` — Styling the Table Cells

The **`tab_style()`** method is a bit more complicated than the previous ones we tried out.

Code

```
from great_tables import GT

GT(<data>) \
  .tab_style(
    style=...,
    locations=...
  )
```

The additional complication is in the ... parts. They require the use of helper functions.

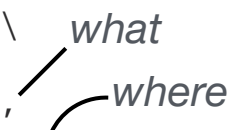
Style: `tab_style()` — Styling the Table Cells

There are two parameters in `tab_style()`: **style=** and **locations=**.

Code

```
from great_tables import GT

GT(<data>) \
  .tab_style(
    style=...,
    locations=...
  )
```



Guide to parameters:

style=: what is the styling we are going to use?

locations=: where is the styling going to be used? Or, which cells receive the styles?

Style: `tab_style()` — Styling the Table Cells

Helpers used to define the **`style=`** and **`locations=`**.

`style=`

`style.fill()`

`style.text()`

`style.borders()`

`locations=` use the **`loc`** class

`loc.body()`

`loc.stub()`

`loc.header()`

`loc.footer()`

*Before
Styling*

num	char
1.111e-01	apricot
2.222e+00	banana
3.333e+01	coconut
4.444e+02	durian
5.550e+03	NA
NA	fig
7.770e+05	grapefruit
8.880e+06	honeydew

*After
Styling* ^{x2}

num	char
1.111e-01	apricot
2.222e+00	banana
3.333e+01	coconut
4.444e+02	durian
5.550e+03	NA
NA	fig
7.770e+05	grapefruit
8.880e+06	honeydew

Style: `tab_style()` — Styling the Table Cells

IMPORTANT: we need two extra imports if using `tab_style()`.

Code

```
from great_tables import GT, style, loc

GT(<data>) \
  .tab_style(
    style = style...,
    locations = loc...
  )
```

Importing **style** and **loc** is important here. Otherwise, you can't use `tab_style()`.

Style: `tab_style()` — Styling the Table Cells

We will go back to the baseline table (using **exibble** for this).

Code

```
GT(exibble)
```

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
	NA fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b

Style: `tab_style()` — Styling the Table Cells

Let's style the entire **num** column with a light blue background color.

Code

```
GT(exibble) \
  .tab_style(
    style=style.fill(color="lightblue"),
    locations=loc.body(columns="num")
  )
```

num	char	fctr	date	time	datetime	currency	row	group
1.111e-01	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.950	row_1	grp_a
2.222e+00	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.950	row_2	grp_a
3.333e+01	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.390	row_3	grp_a
4.444e+02	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.000	row_4	grp_a
5.550e+03	NA	five	2015-05-15	17:55	2018-05-05 04:00	1325.810	row_5	grp_b
NA	fig	six	2015-06-15	NA	2018-06-06 16:11	13.255	row_6	grp_b
7.770e+05	grapefruit	seven	NA	19:10	2018-07-07 05:22	NA	row_7	grp_b
8.880e+06	honeydew	eight	2015-08-15	20:20	NA	0.440	row_8	grp_b